

第 08 章 | 情緒分析與自然語言處理實戰

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線與學習里程碑

導言本章處理非結構化文字資料。你將把文字轉成可供模型使用的數值特徵，建立影評情緒分類器，並比較詞袋、TF-IDF 與雜湊表示法。

本章學習路線本章會串接前面學過的步驟：

架構整合：承接第六章 Pipeline (管線) 概念，將文字前處理與分類模型封裝為可重複使用的流程。

演算法演進：當資料量達到「記憶體無法負荷」的極限時，我們將重新啟用第二章的 SGD (隨機梯度下降) 邏輯，實現增量學習。

本章核心任務：對 IMDb 5 萬筆影評進行二元分類。根據來源定義：評分 > 6 顆星為正評 (1)，評分 < 5 顆星為負評 (0)。我們將挑戰在極高維度的文字空間中，提取出足以區分「讚賞」與「批評」的特徵訊號。

學習目標：完成本章後，你將能夠...

實作 詞袋模型 (Bag-of-Words) 與具備懲罰機制的 TF-IDF 權重計算。

運用 正規表示式 (Regex) 清洗 HTML 雜訊，並採取實務性的表情符號保留策略。

比較 字根還原 (Stemming) 與 詞幹分析 (Lemmatization) 的權衡。

使用 核外學習 (Out-of-core Learning) 與 雜湊特徵 (Hashing Trick) 分批處理大量資料。

利用非監督式的 LDA (主題模型) 在無標籤資料中自動洞察主題結構。

2. 核心觀念：詞袋模型 (Bag-of-Words) 與詞彙權重

電腦如何「空間化」文字？電腦對英文字母一竅不通，它只認識數字。因此，我們必須將文字轉換為數值向量，這稱為「文字空間化」。

核心觀念：計數工讀生與字典索引想像一位「計數工讀生」，他先翻遍所有影評，建立一本詞彙字典 (Vocabulary Dictionary)，為每個唯一的單字編號（索引）。接著，他巡視每篇評論，統計每個編號對應單字出現的次數。這就是詞袋模型：它捨棄了語法與順序，只關心單字是否存在及其頻率。

重要術語對照

中文術語	英文術語	說明
詞袋模型	Bag-of-Words (BoW)	將文本表示為單字頻率向量的模型。
單詞/N-元語法	Token / N-gram	基本單位。1-gram 為單字，2-gram 為連續兩個單字組。
稀疏矩陣	Sparse Matrix	包含大量 0 的矩陣（因單篇評論僅會用到字典的一小部分字）。
詞頻-逆檔案頻率	TF-IDF	衡量單字在特定檔案中重要程度的權重指標。

TF-IDF 的「懲罰機制」與 L2 正規化單純計數是不夠的，因為 "the"、"is" 等字頻率雖高，卻是無意義的「雜訊」。

IDF 的對數縮放：IDF 的公式中使用了對數 (Logarithmic scale)。這項權重設計會降低常見詞的影響，如果一個單字在「所有文章」都出現，其 IDF 值會趨近於 0，降低常見詞對模型的影響。

L2 正規化與單位向量：若評論 A 有 1000 字，評論 B 只有 10 字，A 的詞頻自然會遠高於 B。我們透過 L2 正規化將所有特徵向量縮放為單位向量 (Unit Vector)。無論文章長短，它們最終都在同一個「起跑線」上競爭。

3. 文字前處理：正規表示式與字根還原實戰

資料清洗：訊號優於雜訊在 NLP 中，資料品質決定了模型的上限。未經處理的 HTML 標籤（如 `<br />`）會變成無意義的高頻特徵，嚴重干擾模型判斷。

技術亮點：表情符號的實務價值在實作 `preprocessor` 函數時，我們採取「先提取、後清洗」的策略：

Regex 優先提取：利用 `re.findall` 先把 `:-)`、`:(` 等表情符號抓出來。這非常關鍵，因為若先進行標點符號清洗，這些能直接反應情緒的「強訊號」會被當成普通標點符號刪除。

移至句尾：將表情符號移至清洗後的文字末尾，確保模型能捕捉到這些核心情緒特徵。

字根還原 (Stemming) vs. 詞形還原 (Lemmatization)

Stemming (PorterStemmer)：粗暴但高效的規則切除。它會把 `running` 變回 `run`，但也可能把 `thus` 變成非正規單字 `thu`。這在分類任務中是可接受的，因為它能大幅縮減字典維度。

Lemmatization：更精準的語言學還原（如 `was` 變回 `be`），但計算成本極高。身為顧問，我建議在處理 IMDb 這類大型資料集時，選用「髒但快」的 Stemming 即可。

4. 模型建立：Pipeline 與 GridSearchCV 的常用組合

工程化思維：自動化管線手動調整「要不要拔停用詞」或「C 參數要多少」是低效的。我們使用 `Pipeline` 將 `TfidfVectorizer` 與 `LogisticRegression` 結合，確保特徵工程與模型訓練的原子性。

GridSearchCV 的策略佈局我們將系統化測試以下組合，以找出約 90% 準確率的較佳參數：

特徵選擇：純詞頻 vs. TF-IDF。

前處理：使用自定義斷詞 (`tokenizer`) vs. 字根還原斷詞 (`tokenizer_porter`)。

正規化強度：測試 `C` 參數 (1.0, 10.0, 100.0) 以控制過擬合。

5. 大資料應對：核外學習 (Out-of-core Learning)

技術預警：記憶體崩潰當資料量達到千萬等級，`TfidfVectorizer` 會因為要維護整本「字典」而耗盡記憶體。此時必須轉向「讀一批、學一批、丟一批」的串流邏輯。

HashingVectorizer 的固定維度表示在大資料場景下，我們捨棄 TF-IDF，改用 `HashingVectorizer`。

Stateless (無狀態)：它利用雜湊函數將文字映射到固定維度（如 `2**21`），不需要在記憶體中建立字典。這意味著它不需要預先看過全部資料，是真正「不吃記憶體」的救星。

SGDClassifier (`partial_fit`)：搭配增量學習，讓模型在有限記憶體下分批處理大量資料。

6. 非監督式學習：主題模型 (LDA) 的洞察

非監督式學習的實務地位當面對無標籤資料時，主題模型如同「自動分類器」，能幫我們從混沌中理出頭緒。

⚠ 重要警告 (Critical Warning)：此處的 LDA (Latent Dirichlet Allocation) 是一種生成機率模型 (Generative Probabilistic Model)，用於主題建模。這與第五章學過的「線性判別分析 (Linear Discriminant Analysis)」縮寫相同，但全名與數學原理完全不同。

LDA 運作原理：矩陣分解與 EM 演算法LDA 透過 期望最大化 (Expectation-Maximization, EM) 演算法，將「檔案-單字」矩陣分解為：

檔案-主題矩陣：這篇文章屬於哪些主題？

主題-單字矩陣：這個主題由哪些字（如 horror, house）組成？

透過觀察權重最高的單字，我們可以直覺推斷出主題類別（例如看到 fight, action 即可推論為「動作片」）。

7. 學習指南：Colab 操作步驟、錯誤排除與練習

實作執行建議：NLP 運算量較大，請依照 Colab 的執行進度分段完成。

Colab 操作與風險提醒清單

資料預備：`movie_data.csv` 生成需 10 分鐘，建議直接分發成品檔案。

Windows 多核心臭蟲 (Pickling Error)：

警告：在 Windows 環境下，`GridSearchCV` 設定 `n_jobs=-1` 時，若使用自定義 tokenizer 常會報錯。

解決方案：若你遇到此問題，建議將 `n_jobs` 設為 1，或如需縮短執行時間，可將 tokenizer 替換為內建的 `str.split`。

環境初始化：務必先執行 `nltk.download('stopwords')`。

常見錯誤排除 (FAQ)

Q: 為何跑 `GridSearch` 沒反應？

A: 運算維度過高。Plan B：在等待時，比較 L1 與 L2 正規化在高維文字資料中的稀疏性差異。

Q: `HashingVectorizer` 會發生碰撞嗎？

A: 會，但設定足夠大的 `n_features` (如 `2**21`) 可將碰撞率降至最低，實務上對準確率影響極小。

動手練習與檢查點

理解檢查點：解釋 IDF 公式中的 Log 為何能「平滑」權重？（提示：防止罕見字權重過度膨脹）。

微型實作：修改 Regex，將 `[\W]+` 改為保留底線 `_` 的模式，觀察對分類結果的影響。

自我檢查表：你是否能區分 Stemming 產生的非正規字與 Lemmatization 的差異？

本章總結完成本章後，你能把文字轉成數值特徵、訓練情緒分類器，並依資料量與記憶體限制選擇合適的向量化方法。